



# GEO Meta-Analysis Toolkit

Autonomous Multi-Tool Agent for NCBI GEO Exploration, Metadata Aggregation, & AI Narrative Landscape Synthesis

⚡ 3-Phase Pipeline | 🧠 Local Ollama medgemma:4b | 🔬 NCBI Entrez E-Utils | 🧬 FAISS Vector Index

# Biomedical Discovery Bottlenecks

Why traditional manual search and generic linear scripts fall short in high-throughput genomics.



## Data Scale & Complexity

- NCBI GEO holds >160,000 datasets across thousands of organisms.
- Complex search syntax (Entrez Boolean & Field tags).
- Non-standardized sample descriptions and metadata schemas.



## Manual Synthesis Time

- Extracting organism distributions and timelines takes hours.
- Manual cross-referencing of PubMed literature.
- Prone to human error when handling hundreds of GSE records.



## LLM & Hardware Specs

- Pure agentic ReAct loops stall on LLM reasoning timeouts.
- No Rate limits and API cost overruns.
- Laptops with 8 GB GPU VRAM.

## THE SOLUTION

# Unified AI-Powered Meta-Analysis

Bridging deterministic high-speed computation with autonomous agent reasoning.

**< 2s**

### INSTANT AGGREGATION

Phase 1 direct parsing provides real-time chart rendering before LLM starts.

**100%**

### EXECUTION RESILIENCE

3-phase execution ensures no search query ever fails.

**3.5 GB**

### VRAM EFFICIENCY

Tailored for local GPU acceleration using bio LLM medgemma:4b.

**10,000**

### MAX SCALABILITY

Scalable data fetching engine tuned with NCBI API key rate-limiting.

# Unified Smart Hybrid Architecture

Modular design decoupling user interaction, agent reasoning, API tools, and semantic RAG vector search.

## Streamlit Dark UI Dashboard

Summary Metrics • Visual Charts • Datasets Table • Reasoning Trace

## Unified Smart Hybrid Engine

Phase 1: Direct GEO E-Utils → Phase 2: LangChain ReAct PubMed → Phase 3: Auto Fallback

### Local GPU LLM

Ollama (medgemma:4b)  
Strict Local GPU/CPU Execution

### NCBI E-Utilities

GEO Datasets Search  
PubMed Abstract Retrieval

### FAISS Vector Store

Cell Ontology (CL, UBERON, EFO)  
Two-Pass LLM Normalizer

# 3-Phase Workflow

Eliminating LLM single-point failures through layered execution fallback.

## Phase 1: Direct Engine

⚡ Execution Time: ~2 seconds

- Queries NCBI GEO directly via E-utilities API.
- Parses XML metadata instantly.
- Extracts organism counts, timelines, sample stats, and title terms.

## Phase 2: ReAct AI Synthesis

🧠 Execution Time: ~10-15 seconds

- Spawns specialized LangChain ReAct Agent.
- Autonomously queries PubMed for literature context.
- Synthesizes qualitative research landscape summary.

## Phase 3: Automatic Fallback

🛡️ 100% Uptime Shield

- Catches LLM timeouts, parsing errors, or network issues.
- Seamlessly auto-degrades to linear summarizer.
- Guarantees zero empty outputs on any search query.

# Ontology Disambiguation & RAG

Standardizing fragmented biomedical jargon using FAISS vector search & ontology mappings.



## Supported Biomedical Ontologies

- Cell Ontology (CL): Standardized cell types (e.g., CD8+ T cell, Podocyte, Microglia).
- UBERON: Anatomical structures and tissue locations across species.
- Experimental Factor Ontology (EFO): Experimental assay types, variables, and platform technology.



## Two-Pass Cost-Gated Disambiguation

1. Fast Vector Search: Cosine similarity lookup in pre-computed FAISS index.
2. Cost-Gated LLM Normalizer: Triggers ONLY when similarity is below 0.70 threshold.
  - Resolves ambiguous jargon (e.g., 'MACS' cell separation method vs. 'macs' slang for macrophages).
  - Zero extra LLM round-trips for clean terms.

# Multi-Tab Meta-Analysis Dashboard

Live aggregated analytics across NCBI GEO dataset search results.



## Summary Tab

- Key summary metrics (total datasets, total sample volume, top organism).
- Flowing LLM narrative overview of the research landscape.



## Visual Charts Tab

- Organism distribution donut chart.
- Yearly publication timeline & top title keyword term frequency.



## Datasets Table

- Full sortable results matrix with GSE ID, Title, Organism, Sample count, and Submission date.



## Raw Data & Trace

- Raw JSON response inspection from NCBI E-Utilities API calls.
- Live reasoning trace panel showing agent scratchpad & tool outputs.

# Local GPU & Hardware Optimization

High-efficiency local LLM inference configured for modern AI PCs.

## Workstation Specs

- GPU: Minimum 8 GB VRAM
- RAM: 8 - 32 GB
- OS: Ubuntu 24.04 LTS (CUDA 12)

## Quantized LLM Profiles

- medgemma:4b (~3.5 GB VRAM)  
Default bio-focused model
- gemma2:9b (~5.5 GB VRAM)  
High-reasoning upgrade
- llama3.1:8b (~4.9 GB VRAM)  
General purpose alternative

## Batching & Limits

- Embedding batch size: 512  
(prevents CUDA OOM).
- NCBI E-utils API: 10 req/s (with free API key).
- Max dataset limit: Scalable up to 10,000 results per query.



# Quick Start & Setup Workflow

Simple 5-step terminal deployment for local development and research.

## Terminal Deployment Commands

### # 1. One-time setup script (CUDA, Python venv, dependencies)

```
chmod +x scripts/setup_ubuntu.sh  
./scripts/setup_ubuntu.sh
```

### # 2. Configure environment (.env)

```
cp .env.example .env # Set NCBI_EMAIL='your_email@example.com'
```

### # 3. Pull local GPU LLM model

```
ollama pull medgemma:4b
```

### # 4. Build semantic ontology vector index

```
python scripts/build_index.py
```

### # 5. Launch the interactive dashboard

```
streamlit run app.py
```

# Future Vision & Summary

Empowering bioinformaticians with autonomous research intelligence.



## Key Achievements

- ✓ 100% resilient 3-phase hybrid execution flow.
- ✓ Sub-second statistics aggregation for up to 10,000 datasets.
- ✓ Offline local GPU AI inference with medgemma:4b.
- ✓ FAISS RAG semantic term normalization.



## Future Development Roadmap

- Spatial Transcriptomics & Single-Cell h5ad file parsing.
- Multi-Agent peer review consensus reports.
- Automated differential gene expression cross-comparison.
- PDF / LaTeX automated report generation.

**Thank You! — Questions & Discussion**